

kfold: Explaining and Enabling Configuration-Dependent Linux Builds

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Working draft. The outline remains below as planning notes. The Introduction, Overview, Technique, and Related Work sections are manuscript prose.

Working thesis: kfold uses state-folding symbolic execution to extract configuration-dependent build conditions from Kbuild Makefiles in linear time, avoiding exponential path explosion by folding branch states into word-level propositional formulas and offloading condition satisfiability to SMT solvers. **Basis:** Project source, tests, experiment outputs, evidence files, and DEVT00L_PLAN.md. **Venue and page budget:** FSE, 20 pages for the main paper. References and appendices do not count toward this budget. **Working title:** *kfold: Explaining and Enabling Configuration-Dependent Linux Builds* (the typeset title above); a longer alternative appends “via State-Folding Symbolic Execution”.

Terminology note: Throughout this paper, align terminology with standard Linux/Kbuild usage (Documentation/kbuild/makefiles.rst). Linux developers speak of *configuration options/symbols* (CONFIG_*), *goal definitions / target variables* (obj-y, obj-m), *conditional object selection, composite objects*, and *descending into subdirectories*. In the Linux kernel community, the word “guard” is not used for Make conditions (it refers strictly to header guards or lock guards). In our formal analysis, we use *build condition* for the propositional formula under which an object or directory is compiled, and reserve the term *guard* for formulas internal to the analysis (guarded words, ambient guards, and directory reachability guards such as *B* and *D*). User-facing descriptions of why speak of the *condition chain* rather than a “guard chain”.

Part	Pages
Title, abstract, and front matter	0.5
Introduction	1.5
Overview	2.5
Technique	4.5
Evaluation, including implementation and setup	7.5
Discussion and limitations	1.5
Related work	1.5
Conclusion	0.5
Main paper total	20

Central story

A developer who changes a Linux source file may run a successful build that never compiles that file. Determining why requires analyzing configuration-dependent Kbuild Makefiles and Kconfig constraints. Classic symbolic execution suffers from path explosion ($O(2^k)$ paths for k independent conditionals), and static analyses must choose a representation for conditional variable values that stays small as Makefiles append to them. kfold solves this challenge through *state-folding symbolic execution*: it executes Makefiles in linear time by folding branch environments at conditional join points into word-level propositional build conditions. While state folding yields disjunctive

Author’s Contact Information: Anonymous Author(s), Anonymous Institution, USA.

2026. ACM XXXX-XXXX/2026/9-ART
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

propositional formulas, kfold offloads condition satisfiability and configuration synthesis to an SMT solver (Z3); a whole-tree Linux run issues 82,405 solver checks and finishes in 34 seconds. The resulting build conditions power developer-facing queries to explain build omissions (why) and synthesize valid configurations for modified code (config-for).

1 Introduction

The build system determines which source files reach the compiler. In the Linux kernel and related configurable systems, the Kbuild build system uses Kconfig configuration symbols to select directories and populate goal definitions such as `obj-y`, `obj-m`, `lib-y`, and `extra-y` [5, 6, 10, 11]. In Linux v6.6, thousands of Kconfig files define over ten thousand configuration symbols, and thousands of Makefiles consult them, governing an enormous space of potential build configurations. A concrete build log and configuration file (`.config`) reveal the objects compiled for that single configuration. However, they do not tell a developer why a different source file was omitted, nor which configuration settings would successfully compile it.

This visibility gap has direct consequences for kernel development, continuous integration, and patch testing. A developer modifying a driver or subsystem may run a build that succeeds completely, yet never compiles the modified code because the active configuration quietly excluded the target file. Empirical studies of Linux configuration interactions [1, 24–26, 30, 33] demonstrate that complex configuration dependencies regularly obscure whether options actually affect compiled artifacts. For example, in Linux 4.4.1, the `ATH5K_PCI` setting could be adjusted by users only when its parent `ATH5K` driver was disabled, rendering the option incapable of affecting the compiled driver [30]. To avoid testing illusions and wasted developer effort, maintainers need two actionable capabilities: explaining why a target file was omitted under a given configuration, and synthesizing a minimal configuration change that enables compilation of touched files.

Extracting the exact build conditions that control object compilation is challenging. One cannot simply collect the `CONFIG_` option names that appear in adjacent Makefile lines. Consider the running example in Figure 2, adapted from Linux 6.6 `fs/ext4/Makefile`. Line 2 appends `ext4.o` to `obj-y` when `CONFIG_EXT4_FS=y`. Line 3 then conditionally assigns `ext4-inode-test.o` to `obj-$(CONFIG_EXT4_KUNIT_TESTS)`. When `CONFIG_EXT4_KUNIT_TESTS=y`, this second assignment evaluates to `obj-y := ext4-inode-test.o`, overwriting the previous list and discarding `ext4.o`. The local condition for compiling `ext4.o` as built-in is therefore `CONFIG_EXT4_FS=y ^ ¬(CONFIG_EXT4_KUNIT_TESTS=y)`—a condition determined by a statement that does not name `ext4.o`. Additional conditions arise from nested conditionals, deferred variable expansions, multi-path directory reachability, composite objects whose members are specified in auxiliary variables, and implicit prerequisites of explicit build rules.

Symbolic execution provides a natural foundation for evaluating Makefiles because it tracks the configurations under which variable definitions hold. However, classic symbolic execution forks a new state at every conditional branch [7, 8, 16, 19, 28, 31]. Because Kbuild Makefiles consist of numerous sequential, independent conditional assignments, path-forking executors suffer exponential state explosion. On a benchmark Makefile whose 16 independent conditionals append to an intermediate list that is later added to `obj-y`, path forking generates 524,289 execution states and requires 185 seconds, and 20 such conditionals exceed a five-minute timeout. The pioneering static analysis framework Kmax [13] first made whole-repository Kbuild analysis tractable, laying the foundation for downstream applications such as localizing the configurations that include a program location [15] and Kconfig constraint extraction and unmet-dependency bug finding (KCLAUSE and KISMET [29]). Kmax represents variable values as alternative string definitions tagged with Boolean conditions encoded as Binary Decision Diagrams (BDDs), and it handles the synthetic benchmarks above in under a second. kfold explores a different point in this design space: it

decomposes values into individual words, keeps conditions as solver formulas, and models the Kbuild build semantics (composite members, built-in routes, and rule prerequisites) needed to compare its predictions object-by-object with physical builds.

We introduce kfold, a tool that extracts reusable build conditions for the object paths in a Kbuild repository via *state-folding symbolic execution*, a form of state merging [20, 35] applied to Makefiles in the spirit of variability-aware analysis [21, 34]. kfold is based on the insight that Kbuild target variables operate as sets of independent words. Rather than exploring execution paths by forking (which incurs exponential $O(2^k)$ path explosion) or computing cross products over complete variable strings, kfold executes statements in linear time by attaching a propositional build condition to each individual whitespace-separated word. At conditional join points, kfold folds branch states back into a single environment: when an assignment conditionally overwrites a variable, kfold narrows the build conditions of pre-existing words ($\phi \wedge \neg\psi$), and when branches leave entries untouched, an identity merge prevents formula expansion. This design makes a deliberate algorithmic trade-off: it achieves linear-time symbolic execution over Makefiles while allowing build conditions to become disjunctive propositional formulas. Crucially, kfold offloads satisfiability checking, directory reachability filtering, and minimal configuration synthesis to an SMT solver (Z3 [9]). Real-world build conditions are modular: the whole-tree Linux run issues 82,405 solver checks and completes in 34 seconds including parsing and execution. kfold then leverages these extracted build conditions to answer developer queries: its `why` command walks the chain of conditions behind an object to explain its build verdict, while its `config-for` command combines build conditions with Kconfig dependency cones to synthesize minimal, valid configurations that compile modified source files.

We evaluate kfold against physical builds of five configurable systems: Linux v6.6, BusyBox, Barebox, U-Boot, and coreboot. On four Linux x86 configurations ranging from 489 objects (`tinyconfig`) to 25,949 objects (`allmodconfig`), the object set kfold predicts equals the physical `.o` inventory of each archived build, with no false positives and no misses. On the four smaller systems, precision ranges from 80.9% (coreboot) to 100% (BusyBox). kfold analyzes the Linux x86 tree (2,022 Makefile instances) in 34 seconds and 162 MB of peak memory; on the same four Linux builds, Kmax’s full-tree run takes 5,174 seconds and reaches 56.5–62.6% overlap with the physical inventories. kfold’s `why` command gives the correct verdict for 28 of 28 tested objects. Its `config-for` command returns fragments for 15 non-conflicting multi-file sets and three hand-crafted patches that survive `make olddefconfig` and select every touched object, and it correctly reports the one deliberately conflicting set; six of the proposed objects were also compiled physically.

This paper makes the following contributions:

- (1) **State-Folding Symbolic Execution for Build Systems:** A symbolic execution algorithm for Kbuild Makefiles that achieves linear-time execution by folding branch environments into word-level guarded values, with support for computed names, deferred expansions, conditional overwrites, and fast-path branch merging (§3.3–§3.4).
- (2) **Kbuild Build Semantics and Directory Traversal:** An incremental worklist algorithm for conditional directory reachability, combined with static models of Kbuild build semantics including composite members, built-in routes, and rule prerequisite closures (§3.2, §3.5, §3.6).
- (3) **kfold and Developer-Facing Queries:** An open-source implementation providing why build explanations, Kconfig-aware `config-for` for configuration generation, and a persistent analysis cache (§3.7).
- (4) **Empirical Evaluation:** An evaluation across five configurable projects showing object-level agreement with four archived physical Linux builds, 80.9–100% precision on the four other

```

1 # Adapted from Linux 6.6 fs/ext4/Makefile
2 obj-$(CONFIG_EXT4_FS) += ext4.o
3 obj-$(CONFIG_EXT4_KUNIT_TESTS) := ext4-inode-test.o
4 ifeq ($(CONFIG_FS_VERITY),y)
5     EXTRA := verity
6 else
7     EXTRA := crypto
8 endif
9 obj-y += $(EXTRA).o
10 ext4-inode-test-y += inode-test.o

```

Fig. 2. Input Makefile for the kfold walkthrough, adapted from Linux 6.6 fs/ext4/Makefile. The overwrite and EXTRA branch are constructed for this walkthrough and are not verbatim Linux build rules.

Table 1. Local build conditions for Figure 2 before directory guards are applied. The columns distinguish selection through obj-y and obj-m.

Target or member	Built-in	Module
ext4.o	$E_y \wedge \neg T_y$	$E_m \wedge \neg T_m$
ext4-inode-test.o	T_y	T_m
inode-test.o	T_y	T_m
verity.o	V	—
crypto.o	$\neg V$	—

projects, linear scaling where path forking explodes, a comparison with Kmax, and checked developer query outcomes on synthetic tasks (§4).

2 Overview: from Kbuild assignments to build conditions

Figure 1 separates the analysis that produces build conditions from the developer queries that can use them. Here, a build condition is the formula under which kfold’s Kbuild analysis selects an object path. Given a source tree and project settings, kfold parses the relevant Makefiles, symbolically executes their statements using guarded words, and propagates directory guards before recording each object’s build condition and Makefile origin. The resulting map supports explanations for a supplied .config and configuration proposals for changed files. Figure 2 makes this analysis concrete before we follow those two queries.

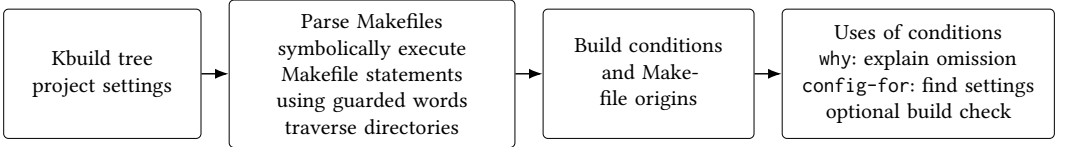


Fig. 1. kfold evaluates Kbuild rules to obtain reusable build conditions and origins. The rightmost box shows optional downstream queries: why explains a build verdict, while config-for proposes a configuration for selected files; a build can check the proposal.

Given a source tree containing the Makefile in Figure 2, kfold derives the built-in and module conditions in Table 1. The Makefile uses option and object names from Linux 6.6 fs/ext4/Makefile, with a constructed overwrite and branch to expose the analysis. Both EXT4 options are tristate, so each can be y, m, or disabled. In a generated Makefile, disabled n expands to an empty value and contributes to neither obj-y nor obj-m. Write E_y and E_m for the two enabled values of CONFIG_EXT4_FS, and T_y and T_m for those of CONFIG_EXT4_KUNIT_TESTS. CONFIG_FS_VERITY is Boolean, so V means it is y and $\neg V$ means it is disabled. Kconfig also defines string, integer, and hexadecimal symbols, although this example uses none of them.

kfold expands the first assignment destination to obj-y under E_y and obj-m under E_m , adding ext4.o to the corresponding list. The second assignment replaces obj-y under T_y and obj-m under T_m , leaving the other list intact in each case. Thus ext4.o survives as built-in under $E_y \wedge \neg T_y$ and

as a module under $E_m \wedge \neg T_m$. The condition for `ext4.o` depends on an assignment that does not name it.

Next, `kfold` evaluates both arms of the conditional and merges `EXTRA=verity` under V with `EXTRA=crypto` under $\neg V$. Expanding $\$(EXTRA).o$ adds `verity.o` or `crypto.o` to `obj-y`, so this assignment has no module route. `kfold` records `inode-test.o` as a member of `ext4-inode-test.o`. The member follows its parent's built-in or module selection.

Table 1 describes target-list selection and composite membership within this Makefile. A selected built-in composite does not necessarily produce a separate object file named after its parent. For a nested Makefile, `kfold` then conjoins built-in selections with the guard B for reaching the directory through a built-in route and module selections with the directory reachability guard D . For example, `ext4.o` has the built-in condition $B \wedge E_y \wedge \neg T_y$ and the module condition $D \wedge E_m \wedge \neg T_m$. Either route can select the object, so its overall build condition is $(B \wedge E_y \wedge \neg T_y) \vee (D \wedge E_m \wedge \neg T_m)$. `kfold` also joins conditions with $\vee$ when separate Makefile assignments or directory paths select the same object. These formulas describe the supported Kbuild analysis, while Kconfig feasibility and physical compilation are checked separately.

These build conditions support the two queries in Figure 1. Given a `.config`, `kfold's why` command evaluates an object's build condition and identifies the first failed part of its condition chain. For example, if B , E_y , and T_y hold, `why` reports that `ext4.o` is not selected because the later assignment replaced its `obj-y` entry. The adapted example does not model Kconfig constraints, so this verdict concerns only its Makefile selection.

Given changed `.c` or `.S` files, or a patch, `kfold's config-for` command maps the touched source files to objects. `kfold` combines their build conditions with the relevant Kconfig constraints and seeks a configuration close to a supplied base configuration. It returns a fragment of option changes and reports files it cannot map or select together. In a recorded Linux task, selecting `fs/btrfs/inode.o` from the saved base configuration required `CONFIG_BTRFS_FS=y`, and the proposed configuration survived `make olddefconfig` and compiled that object. This Linux task supplies the Kconfig and build checks that the adapted Makefile cannot illustrate.

Applying the fragment through `make olddefconfig` checks whether the configurator preserves the requested values. Compiling the selected object checks whether the proposed settings reach a physical build in that task. A satisfying build condition alone establishes neither result, so `kfold` reports the proposal and its verification separately. The next section explains how guarded execution and conditional directory traversal produce the map from object paths to build conditions.

3 Technique

This section describes how `kfold` extracts reusable build conditions from Kbuild Makefiles and uses them to answer developer queries. We first formalize the analysis setting and state the two core algorithms. The remaining subsections explain include splicing, guarded assignments, branch merging, directory reachability, physical build semantics, and query evaluation.

3.1 Setting and Problem Formulation

Let M denote the set of Kbuild Makefiles across a source repository, and let c denote a configuration that assigns values to configuration options $\mathcal{O}$. Each option in $\mathcal{O}$ is Boolean (y or unset), tristate (y, m, or unset), or valued (string, integer, or hexadecimal). We encode a Boolean or tristate option as one propositional variable per enabled value (e.g., E_y and E_m , at most one true), and a Make test that compares a valued option with a literal as an atomic proposition over that option. For an object path o , let $S_M(o, c)$ hold when the build system compiles o under configuration c . `kfold` computes a propositional formula $\Phi_M(o)$ over option variables such that evaluating $\Phi_M(o)$ under c predicts $S_M(o, c)$.

Algorithm 1 Extract guarded Kbuild object conditions

Require: Source tree M , guarded entry files E , project settings Γ

Ensure: Map Φ from object paths to build conditions and origins

```

1:  $R[m] \leftarrow \perp$  for each file  $m \in M$ ;  $Q \leftarrow E$ 
2:  $C \leftarrow$  empty map;  $I \leftarrow \emptyset$ 
3: while  $Q$  is nonempty do
4:   remove  $(m, \gamma)$  from  $Q$ 
5:    $\delta \leftarrow \gamma \wedge \neg R[m]$ 
6:   if  $\delta$  is unsatisfiable then
7:     continue
8:   end if
9:   if  $C[m]$  is undefined then
10:     $C[m] \leftarrow \text{EXEC}(\text{reduce}(\text{splice}(\text{parse}(m))), \sigma_\Gamma, \Gamma)$ 
11:   end if
12:    $S \leftarrow \text{restrict}(C[m], \delta)$ 
13:    $R[m] \leftarrow R[m] \vee \gamma$ ; add  $S$  to  $I$ 
14:   for each selected directory  $(d, \eta)$  in  $S$  do
15:     add  $(\text{entry}(d), \eta)$  to  $Q$  if its entry file exists
16:   end for
17: end while
18:  $\Phi \leftarrow \emptyset$ 
19: for each restricted instance  $S \in I$  do
20:   for each target, member, or program object  $(o, \phi) \in \text{objects}(S, \Gamma)$  do
21:      $\Phi[o] \leftarrow \Phi[o] \vee \phi$  ▷ initially  $\perp$ 
22:   end for
23: end for
24:  $\Phi \leftarrow \text{close}(\Phi, I, \Gamma)$  ▷ resolve composite members, built-in routes, and rules
25: return  $\Phi$ 

```

To compute these formulas without enumerating configurations, *kfold* represents the value of each Make variable as a mapping from whitespace-separated words to build conditions. (Variability and software product line literature often refers to configuration formulas guarding code or assets as presence conditions [3, 6, 22, 34].) Throughout this paper, we consistently use the term *build condition* to denote the propositional formula over configuration options under which a Make variable word, directory route, or compiled object file is selected. Formally, an execution state σ maps each variable name n to a guarded word set $\sigma[n] = \{w \mapsto \phi_w\}$, where ϕ_w is a propositional build condition denoting when word w belongs to variable n . We write $\sigma[n](w) = \perp$ when word w is absent from variable n . For two guarded word sets, $\sigma_1[n] \sqcup \sigma_2[n]$ denotes their pointwise disjunction, where the build condition of each word w becomes $\sigma_1[n](w) \vee \sigma_2[n](w)$.

Algorithm 1 presents the end-to-end extraction procedure. Lines 1–2 initialize the worklist Q with the configured entry files and set the accumulated reachability guard $R[m]$ for every file to false. Lines 3–17 process reachable Makefiles along incoming directory paths. For each work item (m, γ) , line 5 computes the incremental guard $\delta = \gamma \wedge \neg R[m]$ that has not yet been explored for m . If δ is unsatisfiable, lines 6–8 discard the item. Otherwise, lines 9–11 execute the parsed and spliced Makefile under an unrestricted guard when encountering m for the first time, caching the result in $C[m]$. Line 12 restricts the cached state $C[m]$ to the active guard δ , producing an analyzed instance S , and line 13 records S and updates $R[m]$. Lines 14–16 extract downstream directories

Algorithm 2 EXEC: execute one statement over guarded words**Require:** Statement s , state σ , ambient guard g **Ensure:** Updated state σ

```

1: if  $s$  is an assignment  $x \text{ op } e$  then
2:    $V \leftarrow \{w \mapsto \phi_w\}$  from  $\text{expand}(e, \sigma)$ , or  $\{e \mapsto \top\}$  if  $\text{op}$  defers  $e$  and  $e$  contains a reference
3:   for each guarded variable name  $(n, \psi_n) \in \text{expand}(x, \sigma)$  do
4:      $\psi \leftarrow g \wedge \psi_n$ 
5:     if  $\text{op} \in \{=, :=\}$  or  $\sigma[n]$  is undefined then
6:        $\sigma[n] \leftarrow \{w \mapsto \psi \wedge \phi_w\} \sqcup \{w \mapsto \phi \wedge \neg\psi : (w \mapsto \phi) \in \sigma[n]\}$ 
7:     else  $\triangleright \text{op} = +=$ 
8:        $\sigma[n] \leftarrow \sigma[n] \sqcup \{w \mapsto \psi \wedge \phi_w\}$ 
9:     end if
10:  end for
11: else if  $s$  is a conditional with test  $t$ , then-arm  $T$ , else-arm  $F$  then
12:    $\kappa \leftarrow \text{cond}(t, \sigma)$ ;  $\sigma_t \leftarrow \text{copy of } \sigma$ ;  $\sigma_e \leftarrow \text{copy of } \sigma$ 
13:   EXEC( $T, \sigma_t, g \wedge \kappa$ ); EXEC( $F, \sigma_e, g \wedge \neg\kappa$ )
14:   for each variable name  $n$  assigned in  $T$  or  $F$  do
15:     if  $\sigma_t[n](w) = \sigma_e[n](w) = \phi$  then
16:        $\sigma[n](w) \leftarrow g \wedge \phi$   $\triangleright$  fast-path merge for untouched words
17:     else
18:        $\sigma[n](w) \leftarrow (g \wedge \kappa \wedge \sigma_t[n](w)) \vee (g \wedge \neg\kappa \wedge \sigma_e[n](w))$ 
19:     end if
20:   end for
21: end if
22: return  $\sigma$ 

```

selected in S and queue their entry Makefiles under their respective directory guards η . After the traversal terminates, lines 18–23 aggregate target words across all recorded instances. Finally, line 24 computes the transitive closure over composite members, built-in directory reachability constraints, and rule prerequisites before returning the completed map Φ .

Algorithm 2 defines the symbolic execution routine EXEC for individual Make statements. Lines 1–10 handle variable assignments by expanding both the destination variable name and the assigned expression into guarded words. Lines 11–20 handle conditionals by evaluating the test condition, executing both branches on separate copies of the state, and merging modified variables back into σ . We now examine each stage of this analysis in detail.

3.2 Parsing, Include Splicing, and Dependency Reduction

kfold reads each Makefile and parses its syntax with `pymake3`. In Kbuild, an `include` directive evaluates a referenced file within the environment of the including Makefile, using the repository root as its working directory. kfold mirrors this behavior through *static include splicing*: it resolves the path of each included file and substitutes the directive with the parsed statements of the target file before running symbolic execution. Path resolution evaluates standard Kbuild variables such as `src`, `obj`, and `srctree`, as well as constant assignments and string functions such as `addprefix` and `addsuffix`. For example, the Linux AMD GPU driver defines `FULL_AMD_PATH := $(srctree)/$(src)/..` and uses `addprefix` to include 51 display-library Makefiles. kfold resolves 742 composite-member objects under `drivers/gpu/drm/amd/`; without splicing it finds only 240

of them. When an include path depends on dynamic shell outputs or unresolvable variables, `kfold` records the unmerged include and continues analysis.

After splicing includes, `kfold` performs static dependency reduction. Many statements in `Kbuild` Makefiles manipulate compiler flags (such as `ccflags-y` and `CFLAGS`) that do not affect which object files are selected. `kfold` computes the backward dependency cone of all target-affecting variables, composite lists, subdirectories, and build rules. It then removes statements that do not influence these variables before invoking `EXEC`. This reduction discards irrelevant flag manipulations, keeping subsequent symbolic execution focused on target selection.

3.3 Guarded Assignment and Name Expansion

`Kbuild` Makefiles construct target lists by appending and assigning to variables whose names and values contain option references. Algorithm 2 evaluates an assignment $x \text{ op } e$ by expanding the left-hand name x into a set of guarded variable names $\{(n, \psi_n)\}$ and the right-hand expression e into a set of guarded words $\{(w, \phi_w)\}$. The effective guard for writing word w into variable n is $\psi = g \wedge \psi_n \wedge \phi_w$, where g is the ambient execution guard.

When an assignment appends to a variable ($\text{op} = +=$), line 8 joins the new guarded words into $\sigma[n]$ using pointwise disjunction $\sqcup$. When an assignment overwrites a variable ($\text{op} \in \{=, :=\}$), line 6 updates the state conditionally. The assigned words receive the active guard $\psi \wedge \phi_w$. Crucially, any pre-existing word $(w \mapsto \phi)$ in $\sigma[n]$ survives under the narrowed condition $\phi \wedge \neg\psi$, reflecting that the overwrite replaced the previous definition only in configurations where ψ holds. A conditional default ($?=$) leaves existing words untouched and adds each new word under $\psi \wedge \phi_w \wedge \neg \bigvee_{w'} \sigma[n](w')$, that is, only where the variable holds no words; Algorithm 2 omits this case for brevity.

Consider the following pair of `Kbuild` assignments:

```
1 obj-$(CONFIG_A) += a.o
2 obj-$(CONFIG_B) := b.o
```

Assuming an initially empty state $\sigma[\text{obj-y}] = \emptyset$, line 1 expands the destination to `obj-y` under $A_y = (\text{CONFIG_A}=y)$. Appending `a.o` yields $\sigma[\text{obj-y}] = \{a.o \mapsto A_y\}$. Line 2 then expands its destination to `obj-y` under $B_y = (\text{CONFIG_B}=y)$. Because line 2 is an overwrite (`:=`), `kfold` assigns `b.o` under B_y and narrows the existing entry `a.o` by $\neg B_y$, producing the updated state $\sigma[\text{obj-y}] = \{a.o \mapsto A_y \wedge \neg B_y, b.o \mapsto B_y\}$. The build condition of `a.o` accurately captures that `b.o` replaces it whenever B_y holds.

`kfold` models GNU Make variable expansion flavors. For simply expanded assignments (`:=`), `kfold` expands the right-hand expression immediately during execution. For recursively expanded assignments (`=`) and conditional defaults (`?=`), `kfold` stores unexpanded expression text containing variable references and defers evaluation until the end of the Makefile. This deferred expansion matches `Kbuild` semantics, because `Kbuild` reads final target variables only after processing the complete file. For example, in Linux's `drivers/xen/Makefile`, the assignment `dom0-y += $(xen-pad-y)` precedes the definition of `xen-pad-y`; deferred evaluation correctly resolves the reference to include `xen-acpi-pad.o`. Table 2 summarizes the supported Make constructs and their operational boundaries.

3.4 Conditional Execution and Branch Merging

`Kbuild` Makefiles make extensive use of conditional directives (`ifeq`, `ifneq`, `ifdef`, and `ifndef`). When encountering a conditional statement, Algorithm 2 evaluates the test expression t under the current state σ to produce a guard formula κ . For equality tests, `kfold` expands both sides

Table 2. Kbuild Make constructs and kfold's modeled semantics.

Construct	Modeled Semantics	Boundary
<code>+=, =, :=, ::=</code>	Guarded words; GNU Make flavor handling (<code>:= as :=</code>); deferred target lists expanded at file end	Word ordering and duplicate occurrences are not preserved.
<code>?=</code>	Assigned under the configurations in which the variable holds no words	Differs from GNU Make when a variable is defined but empty.
<code>ifeq, ifneq, ifdef, ifndef</code>	Evaluates condition guard, executes both arms on copies, merges modified variables	Tests inherit the limits of variable references below.
Variable references	Guarded words plus empty string alternative; substitution references per word	Concatenation collapses when alternatives exceed 64 combinations.
Text functions	Handled symbolically for <code>filter</code> , <code>patsubst</code> , <code>addprefix</code> , <code>addsuffix</code> , <code>word</code>	Arbitrary custom recursive functions are unrolled to a fixed depth.
<code>include</code>	Recursively spliced when target path resolves statically	Unresolved dynamic include paths contribute no statements.
Rules and sub-makes	Targets and prerequisites captured as guarded lists; followed to closure	Recipes other than sub-makes are not executed.
<code>\$(shell ...)</code>	Executed as concrete subprocess with timeout; empty in sandbox mode	Output reflects the host environment and installed tools.

into guarded strings and defines κ as the disjunction of guards where the two sides coincide. For example, testing `ifeq $(CONFIG_FS_VERITY),y` produces $\kappa = V$.

Rather than forking the entire execution path into separate analysis branches, kfold creates copies of the state σ_t and σ_e for the then-arm and else-arm (line 12). It executes the then-arm T under ambient guard $g \wedge \kappa$ and the else-arm F under $g \wedge \neg\kappa$ (line 13). Lines 14–19 then merge modified variables back into σ . For each variable n assigned in either branch, an entry present with guard ϕ_t in the then-branch and ϕ_e in the else-branch receives the combined guard $(g \wedge \kappa \wedge \phi_t) \vee (g \wedge \neg\kappa \wedge \phi_e)$. Conjoining the ambient guard g is safe inside nested conditionals: the enclosing merge re-gates the whole arm with its own branch guard and restores the configurations outside g from the other arm.

To illustrate the contrast between path forking and kfold's state folding, consider sequential conditionals:

```

1 ifeq $(CONFIG_A),y
2   obj-y += a.o
3 endif
4 ifeq $(CONFIG_B),y
5   obj-y += b.o
6 endif

```

A classic path-forking symbolic executor forks at each conditional, maintaining $2^2 = 4$ separate path environments: $\pi_1(A \wedge B) \mapsto \{a.o, b.o\}$, $\pi_2(A \wedge \neg B) \mapsto \{a.o\}$, $\pi_3(\neg A \wedge B) \mapsto \{b.o\}$, and $\pi_4(\neg A \wedge \neg B) \mapsto \emptyset$. For k sequential blocks, path forking produces $O(2^k)$ execution paths.

In contrast, kfold folds branch states at each `endif` in linear time. After block 1, kfold holds the single folded state $\sigma[\text{obj-y}] = \{a.o \mapsto A\}$. When evaluating block 2, the then-arm produces $\sigma_t[\text{obj-y}] = \{a.o \mapsto A, b.o \mapsto B\}$ under ambient guard B , while the else-arm produces $\sigma_e[\text{obj-y}] = \{a.o \mapsto A\}$ under $\neg B$. At the join point, kfold observes that `a.o` carries the exact same guard A in both arms ($\sigma_t[\text{obj-y}](a.o) = \sigma_e[\text{obj-y}](a.o) = A$). Line 16 applies the fast-path merge, setting the guard of `a.o` directly to $g \wedge A = A$ (here $g = \top$) rather than creating the redundant disjunction $(B \wedge A) \vee (\neg B \wedge A)$. The final state is $\{a.o \mapsto A, b.o \mapsto B\}$, computed in $O(k)$ linear time without path explosion. The identity $(g \wedge \kappa \wedge \phi) \vee (g \wedge \neg\kappa \wedge \phi) \equiv g \wedge \phi$ makes this simplification exact.

3.5 Conditional Directory Reachability

Kbuild orchestrates large source trees hierarchically: a parent Makefile descends into subdirectories listed in target variables and subdirectory lists (`obj-y`, `obj-m`, `subdir-y`, and `subdir-m`). Algorithm 1 explores this directory structure using a worklist Q starting from top-level entry files E .

A naive traversal using a Boolean visited set would fail when the same directory is reachable through multiple independent paths with different configuration guards. Conversely, re-executing a Makefile on every visit would lead to redundant work. `kfold` resolves this trade-off by maintaining an accumulated reachability guard $R[m]$ for each Makefile m . When a work item (m, γ) is dequeued, line 5 computes the uncovered guard $\delta = \gamma \wedge \neg R[m]$. If Z3 determines that δ is unsatisfiable, the visit is redundant and skipped. Otherwise, lines 9–11 obtain the cached baseline execution state $C[m]$ and line 12 restricts it by conjoining all guards with δ . Line 13 updates $R[m] \leftarrow R[m] \vee \gamma$. Lines 14–16 then extract directory targets and append child Makefiles to Q alongside their reachability guards η .

Consider a top-level Makefile that reaches a subdirectory through two independent option routes:

```
1 obj-$(CONFIG_NET) += drivers/net/
2 obj-$(CONFIG_PCI) += drivers/net/
```

When processing the first entry under guard N (for `CONFIG_NET=y`), `kfold` executes `drivers/net/` Makefile once under $\top$, caches its execution state $C[\text{net}]$, restricts it to N , and records $R[\text{net}] = N$. When the second entry arrives under guard P (for `CONFIG_PCI=y`), `kfold` computes $\delta = P \wedge \neg N$. If Z3 determines that δ is satisfiable, `kfold` restricts the cached state $C[\text{net}]$ by δ , updates $R[\text{net}] = N \vee P$, and queues newly reachable subdirectories. If P already implies N , δ is unsatisfiable and `kfold` skips the visit entirely. Because the source repository contains a finite number of Makefiles and option domains are finite, $R[m]$ can strictly weaken only finitely many times, guaranteeing termination.

3.6 Object Extraction and Kbuild Build Semantics

After the directory worklist terminates, `kfold` extracts physical object paths from the recorded instances I . Target extraction inspects standard Kbuild target families (`obj-`, `lib-`, `extra-`, and `always-`) as well as project-specific families (such as `pbl-` in Barebox and `bootblock-` in coreboot). `kfold` models four core build semantics from Kbuild’s `scripts/Makefile.build`:

Composite Object Resolution. When a selected target t represents a composite object, Kbuild compiles its constituent members rather than a single source file of the same name. `kfold` inspects composite definitions in variables such as $t-y$, $t-objs$, $t-lib$, and $t-m$. It assigns each member object the conjunction of its own guard and the container object’s guard. In Linux v6.6, composite members account for 14,351 of the 29,245 analyzed object paths.

Built-in Containers and Objects. When a composite target is selected for a built-in build (`obj-y`), Kbuild links its member objects directly into the directory’s archive (`built-in.a`) and produces no standalone object file for the container name. Only module composites (`obj-m`) generate container object files on disk. `kfold` therefore suppresses built-in composite container paths from the final object map while preserving their member objects.

Built-in Directory Reachability Routes. Kbuild sets an internal flag `need-built-in` during recursive builds: a directory builds its `built-in.a` archive and its `obj-y` targets only if the parent directory was entered through an `obj-y` reference. If a directory is reached solely through an `obj-m` or `subdir-y` reference, Kbuild builds its modules and auxiliary targets but skips its `obj-y` objects.

kfold computes the built-in reachability guard B_m for every Makefile m as a fixpoint over obj-y directory edges and conjoins B_m with every built-in object guard.

Rule Prerequisites and Sub-makes. Certain kernel objects do not appear in target lists and are built only when invoked as prerequisites of explicit or pattern rules. For example, vDSO objects and boot images are compiled through architecture-specific rules in arch/x86/entry/vdso/Makefile and arch/x86/boot/Makefile. kfold records explicit rules, pattern rules, and sub-make invocations of the form \$(MAKE) \$(build)=dir goal as guarded structures. Line 24 of Algorithm 1 computes a prerequisite closure starting from all selected objects and configured goals, pulling in rule prerequisites under the conjunction of their triggering guards.

3.7 Applications: Developer Queries and Analysis Cache

The resulting map Φ stores the exact build conditions and origin Makefiles for all discovered objects. kfold uses this map to answer developer queries about the project build without re-analyzing the source tree.

Explaining Omissions (why). Given a configuration file .config and a target object path o , kfold evaluates $\Phi(o)$ under the supplied configuration. If the condition evaluates to false, kfold traverses the chain of conditions that produced $\Phi(o)$ —including directory reachability, assignment guards, and composite membership—and identifies the first unsatisfied guard term. This pinpoints whether the object was omitted due to an unset top-level option, a directory bypass, or an overwrite in a specific Makefile.

Enabling Changed Files (config-for). Given a set of modified source files or a patch, kfold maps each source file to its corresponding object path o and retrieves build condition $\Phi(o)$. To ensure that the requested options can be enabled without violating kernel dependencies, kfold extracts the Kconfig dependency cone for all options appearing in $\Phi(o)$. It translates both the build conditions and the Kconfig dependencies into a propositional formula and queries Z3 to find a satisfying assignment that minimizes changes relative to a base .config. If the targets have conflicting requirements, kfold reports the mutually incompatible file subsets.

Analysis Cache and Invalidation. kfold persists each object’s kind, origins, and build condition, together with the reachability conditions of analyzed Makefiles, in a per-tree cache: a JSON metadata file plus the conditions as SMT-LIB terms with shared subterms. Subsequent invocations of why and config-for load this cache and parse conditions lazily, one object at a time. The cache records the path, modification time, and size of every Makefile, included fragment, and settings file the analysis read, plus a content hash of the analyzer itself. If any of these changes, kfold discards the cache and reruns the whole-tree analysis; incremental re-analysis is future work.

4 Evaluation

This section evaluates kfold across five configurable software projects: the Linux kernel (v6.6), BusyBox (1.36.1), Barebox (2024.01.0), Das U-Boot (2024.01), and coreboot (4.22.01). Our evaluation investigates four research questions:

- **RQ1 (Build Agreement):** How accurately do kfold’s extracted build conditions predict compiled object files across concrete configurations and diverse Kbuild dialects? (§4.2)
- **RQ2 (Scalability and State Folding):** Does state-folding symbolic execution avoid the state explosion of path-forking symbolic execution, and how fast is whole-tree analysis? (§4.3)

Table 3. Evaluated subject systems, configurations, and build provenance.

Subject	Version / Commit	Tested Configuration	Physical Objects
Linux	v6.6 (ffc25326)	i386 tinyconfig	489
Linux	v6.6 (ffc25326)	x86-64 defconfig	2,874
Linux	v6.6 (ffc25326)	Debian x86-64	15,419
Linux	v6.6 (ffc25326)	x86-64 allmodconfig	25,949
Barebox	2024.01.0	sandbox_defconfig	406
Das U-Boot	2024.01	sandbox_defconfig (archived / fresh)	1,123 / 1,309
BusyBox	1.36.1	defconfig	191
coreboot	4.22.01	QEMU i440fx	674

- **RQ3 (Developer Diagnostics and Automated Configuration):** How effectively do kfold’s why and config-for queries explain build omissions and synthesize configuration fragments for modified source code? (§4.4)
- **RQ4 (Comparative Analysis and Ablations):** How does kfold compare with Kmax on the same physical builds, and which Kbuild modeling features account for its agreement? (§4.5)

4.1 Experimental Setup and Provenance

Table 3 summarizes the evaluated subjects, their tested configurations, and their physical artifact status. For Linux, we evaluate four x86 configurations spanning minimal to exhaustive profiles: i386 tinyconfig (489 compiled objects), x86-64 defconfig (2,874 compiled objects), Debian x86-64 distribution configuration (15,419 compiled objects), and x86-64 allmodconfig (25,949 compiled objects). Clean rebuilds using GCC 12.5.0 reproduced the exact physical object inventories for Linux tinyconfig, Linux defconfig, and Barebox sandbox_defconfig. The Debian and allmodconfig Linux comparisons, BusyBox, coreboot, and one U-Boot comparison use archived build inventories; a second, fresh U-Boot build completed compilation but failed during final packaging. The Linux archives omit tools/ and scripts/; predictions under top-level directories absent from an archive are reported separately rather than counted as false positives (one object, under allmodconfig).

All experiments were executed on an Intel Core i7-7700 workstation equipped with 8 logical CPUs and 31 GiB of RAM running Debian Linux. The analysis environment ran Python 3.14.7, Z3 4.13.3, and GNU Make 4.4.1. Exact source repository commits and configuration SHA-256 hashes are recorded in evidence/manifest.json.

4.2 RQ1: Build Agreement

To evaluate agreement, we compare kfold’s predicted object sets against concrete physical build outputs. Let U denote the set of extracted object paths (targets, composite members, programs, and rule prerequisites), and let $P_c \subseteq U$ denote the predicted object set obtained by evaluating all build conditions under configuration c . Let B_c denote the ground-truth inventory of physical .o files compiled on disk under c . We partition the comparison into true positives $TP = |P_c \cap B_c|$, false positives $FP = |P_c \setminus B_c|$, misses within the extracted set $FN_U = |(B_c \cap U) \setminus P_c|$, and physical objects outside the extracted set $O = |B_c \setminus U|$. Precision is defined as $TP/(TP + FP)$, while whole-inventory overlap is defined as $TP/|B_c|$.

Table 4 reports the physical build comparison for Linux v6.6. On all four profiles, the predicted set equals the physical .o inventory: every compiled object is predicted and every predicted object is compiled. The extracted set U holds 29,245 object paths, of which 24,596–29,233 fall inside the top-level directories each archive covers. Composite members are essential to this result. Counting only the words of target-family variables, kfold’s predictions cover 38.2–88.5% of each inventory,

Table 4. Linux physical build agreement across four configuration profiles. The last column gives overlap when composite members are not resolved (targets only).

Configuration	Pred. ($ P_c $)	TP	FP	FN_U	O	Prec.	Overlap	Targets only
tinyconfig (i386)	489	489	0	0	0	100%	100%	88.5%
defconfig (x86-64)	2,874	2,874	0	0	0	100%	100%	49.4%
Debian (x86-64)	15,419	15,419	0	0	0	100%	100%	38.2%
allmodconfig (x86-64)	25,949	25,949	0	0	0	100%	100%	47.4%

Table 5. Physical build agreement for the four non-Linux subjects.

Subject	Pred. ($ P_c $)	TP	FP	FN_U	O	Prec.	Overlap
BusyBox	162	162	0	0	29	100.0%	84.8%
Barebox	406	402	4	0	4	99.0%	99.0%
Das U-Boot (archived)	1,029	952	77	0	171	92.5%	84.8%
Das U-Boot (fresh)	1,029	1,029	0	0	280	100.0%	78.6%
coreboot	540	437	103	0	237	80.9%	64.8%

because most compiled objects of built-in and module composites never appear in a target list themselves.

Table 5 presents physical agreement across the four other projects. No subject has a miss inside the extracted set. On BusyBox, kfold achieves 100% precision (162/162); all 29 outside objects are intermediate built-in.o archives, which older Kbuild dialects produce as .o files. On Barebox, precision and overlap are both 99.0%; the four outside objects are generated image metadata and embedded default-environment blobs. On U-Boot, the 77 false positives against the archived build (41 under lib/ and 36 under cmd/) disappear against the fresh build, which suggests that the archived inventory and configuration are out of step; most outside objects are driver objects (121) selected outside the analyzed Makefile traversal, and the fresh build adds host tools and generated files. On coreboot, precision is 80.9%: all 103 false positives lie under the stage-specific build directories verstage, smm, and decompressor, reflecting coreboot’s multi-stage compilation model, and most outside objects are host utilities (build/util) and external payloads.

4.3 RQ2: Scalability and State Folding vs. Path Forking

To isolate the effect of state folding, we generated Makefiles with k independent options in four families (tools/bench_scaling.py): *append* (obj-\$(CONFIG_A_{*i*}) += a_{*i*}.o), *ifeq* (an ifeq block that appends b_{*i*}.o to obj-y), *value* (an ifeq/else that sets V_{*i*}, followed by obj-y += c_{*i*}\$(V_{*i*}).o), and *indirect* (ifeq blocks that append d_{*i*}.o to an intermediate OBJS, followed by obj-y += \$(OBJS)). We compare kfold with a path-forking executor built from the same code base (commit a3c1683) and with Kmax, under a five-minute timeout.

Table 6 reports the results. The path-forking executor’s state count stays linear for *append* (2k states; 256 at $k = 128$) and for *ifeq* ($k + 1$ states), because independent appends to the target list can be joined without enumerating combinations; its time on *ifeq* still grows superlinearly and exceeds the timeout at $k = 128$. When branches write values that are later combined, forking explodes exponentially: *indirect* reaches 524,289 states and 184.6 s at $k = 16$ and times out at $k = 20$, and *value* reaches 49,152 states at $k = 12$ and times out at $k = 16$. kfold’s time is flat at 0.61–0.73 s for every family up to $k = 128$ and dominated by start-up, and its object sets match the expected sets exactly in every run. Kmax is also fast on all four families (0.67–0.90 s), so this benchmark separates

Table 6. Scaling on generated Makefiles: path-forking states and seconds per family, and the slowest kfold and Kmax time over all four families. All times include process start-up (about 0.5 s). “t/o” marks the five-minute timeout and “–” a run skipped after a smaller k timed out.

k	Path forking: states / time (s)			kfold	Kmax
	<i>ifeq</i>	<i>value</i>	<i>indirect</i>	time (s)	time (s)
4	5 / 0.59	64 / 0.62	33 / 0.53	0.63	0.69
8	9 / 0.71	2,048 / 3.18	1,025 / 0.86	0.63	0.69
12	13 / 0.96	49,152 / 75.51	24,577 / 8.65	0.62	0.79
16	17 / 1.50	t/o	524,289 / 184.59	0.62	0.70
20	21 / 2.54	–	t/o	0.63	0.71
64	65 / 265.79	–	–	0.62	0.79
128	t/o	–	–	0.73	0.90

Table 7. End-to-end extraction performance on the five repositories (results/physical_validation.json).

Subject	Makefile instances	Targets	All objects	Time (s)	Peak RSS (MB)
BusyBox	29	609	609	0.27	65.6
Barebox	160	1,363	1,436	0.96	91.6
Das U-Boot	289	3,133	3,209	3.02	94.7
coreboot	458	4,594	4,594	7.98	90.5
Linux x86	2,022	14,793	29,245	34.05	161.7

state folding from path forking, not kfold from Kmax. On the *value* family, Kmax reports $3k$ objects where $2k$ are expected.

Table 7 reports end-to-end extraction runtime and object volume across all five repositories. For the Linux x86 tree, kfold parses and symbolically executes 2,022 Makefile instances, issues 82,405 solver checks, and extracts build conditions for 29,245 object paths (14,793 target words plus composite members, programs, and rule prerequisites) in 34.05 seconds with 161.7 MB of peak resident memory. The four other repositories complete extraction in under 8 seconds. Whole-tree analysis is therefore fast enough to run once per tree revision and reuse through the analysis cache, as the developer queries in RQ3 do.

4.4 RQ3: Developer Diagnostics and Automated Configuration

We evaluate kfold’s developer-facing commands on Linux maintenance tasks.

Explaining Omissions (why). We evaluated kfold’s why command on 28 object targets across Linux defconfig and tinyconfig. For all 28 targets, why produced build verdicts matching the physical build inventories. For excluded targets, why pinpointed the failed part of the condition chain, for example an unset top-level option (CONFIG_BTRFS_FS unset in defconfig) or an unsatisfied directory reachability route (drivers/net/ not entered under tinyconfig).

Synthesizing Configurations for Modified Files (config-for). We tested config-for against the Linux defconfig base on 16 synthetic multi-file sets drawn from eight subsystems and on three hand-crafted patches (evidence/config_for_checkpoint_2b.json). The working tree has a single squashed commit, so no real patch history was available. The sets mix built-in objects that need no change, module-capable objects, composite members, objects whose options are

off in `defconfig` (e.g., `fs/btrfs/inode.o`, which needs `CONFIG_BTRFS_FS=y`), and one deliberate Kconfig conflict (`entry_32.o` versus `entry_64.o`). For all 15 non-conflicting sets, every requested object is predicted to be built, and applying the fragments through `make olddefconfig` loses no requested symbol, both in a combined 15-object run and in a 3-object run with compilation. For the conflicting set, `config-for` excludes exactly one of the two mutually exclusive objects, gives the correct reason, and builds the other. For all three patches, every touched `.c` file maps to an object, touched headers are reported as not directly compiled, and the verified configuration keeps every requested symbol and selects every touched object. We additionally compiled six of the proposed objects physically (including `fs/ext2/xattr.o`, `fs/btrfs/inode.o`, and `drivers/net/ethernet/intel/e1000e/netdev.o`); all six compiled. A typical query takes 1.5–2.8 s, dominated by parsing the Linux Kconfig tree.

Configuration Blind Spots Analysis. Using `kfold`'s whole-tree build condition map, we audited configuration blind spots in Linux v6.6. We queried which of the 29,245 analyzed object paths are selected by none of three generated x86 configurations (`allmodconfig`, `allyesconfig`, and `defconfig`). The query identified 3,242 objects (11.1%) that no configuration selects, spanning 493 entries of the Linux MAINTAINERS file. `kfold` attributes 251 of them to options of non-x86 architectures (e.g., `CONFIG_S390`); many of the rest need options that none of the three configurations enables, such as platform options (`CONFIG_ARCH_SUNXI`, `CONFIG_ARCH_HISI`) or options excluded by other choices (`CONFIG_PREEMPT_RT`). The query shows how build conditions let maintainers see which code the usual build configurations never compile.

4.5 RQ4: Comparative Analysis and Ablations

Comparison with Kmax. We ran `Kmax` as shipped (`kmaxall-T-DSRCARCH=x86`) over the Linux tree and evaluated its conditions against the same four physical builds (`results/kmax_physical_validation.json`). `Kmax`'s run took 5,174 s of wall time, against 34 s for `kfold`; its peak memory was not recorded. Evaluating `Kmax`'s per-Makefile conditions conjoined with its directory conditions, `Kmax` achieves 82.6–97.6% precision and 56.5–62.6% overlap with the four Linux inventories, and it misses 125–7,902 compiled objects that are inside its extracted universe. Using per-Makefile conditions alone raises overlap to 58.7–81.2% but lowers precision to 31.7–96.5%. On `BusyBox`, both tools extract the same 609 objects and reach the same 100% precision and 84.8% overlap; `Kmax` takes 25.6 s. The main differences on Linux come from the Kbuild semantics that `kfold` models on top of its conditions (composite members, built-in routes, and rule prerequisites), which the ablations below isolate.

Ablation Study. We disabled one modeling feature at a time and repeated the Linux physical comparison (`results/ablation.json`):

- **Composite Object Resolution:** Without composite members, the extracted set shrinks from 29,245 to 14,894 object paths, losing 14,351 members (49.1%), and overlap with the four inventories drops from 100% to 38.2–88.5% (Table 4).
- **Static Include Splicing:** Without splicing, `kfold` loses 1,223 composite members, including 502 of the 742 members under `drivers/gpu/drm/amd/`. Precision stays at 100%, but 1,223 Debian and `allmodconfig` objects fall outside the extracted set, lowering overlap to 92.1% and 95.3%.
- **Rule Prerequisite Closures:** Without rule prerequisites, `kfold` loses 88 objects, 63 of them under `arch/x86/entry/vdso/` and `arch/x86/boot/`; overlap drops to 91.6% on `tinyconfig` and to 97.8–99.7% on the other profiles.

- **Built-in Routes and Container Suppression:** Ignoring need-builtin routes and built-in container suppression keeps every compiled object but introduces 5–148 false positives per profile (precision 95.1–99.6%).
- **Entry-Point Settings:** With the entry-point settings used before this study (1,952 Makefile instances), overlap falls to 43.4% on tinyconfig and 81.8–96.0% on the other profiles, with up to five misses.

4.6 Threats to Validity

Construct Validity. Our evaluation measures agreement between predicted object paths and physical build artifacts. Physical inventories contain `.o` files that are not ordinary compilation units (for example BusyBox’s `built-in.o` archives, generated blobs, and host tools), and the Linux archives omit `tools/` and `scripts/`. We explicitly distinguish true positives, misses inside the extracted set, and outside objects in all reported tables, and report predictions outside an archive’s scope separately. Four Linux builds, all x86, cannot show agreement on every configuration; exact agreement on these four is evidence, not a proof of soundness.

Internal Validity. Our analysis relies on the `pymake3` parser and Z3 SMT solver. Bugs in parser grammar or constraint encoding could affect extracted formulas. To mitigate this threat, we compared the formulas’ predictions with physical GCC builds under four Linux and five non-Linux configurations, checked every `config-for` fragment in RQ3 through `make olddefconfig`, and compiled six proposed objects.

External Validity. Although Linux is our primary subject, we evaluated four additional configurable code bases: the BusyBox userspace utilities and the Barebox, U-Boot, and coreboot firmware projects. While these subjects demonstrate broad applicability across Kbuild dialects, build conventions in other non-Kbuild ecosystems may differ.

5 Discussion and Limitations

5.1 What Build Conditions Support

The primary utility of `kfold`’s build conditions is providing transparent, queryable explanations of build-system behavior. When an object is unexpectedly omitted from a build, maintainers can inspect its build condition to trace the exact chain of configuration requirements, directory reachability guards, and assignment overwrites responsible for the omission. Furthermore, `kfold` enables proactive configuration synthesis by translating build conditions into SMT constraints, allowing developers to generate configuration deltas for newly modified files without manual trial and error.

Maintainers should note that a Kbuild-local build condition $\Phi(o)$ denotes the formula under which Makefiles select object o . It does not guarantee that $\Phi(o)$ is satisfiable under global Kconfig rules. For this reason, `kfold` combines build conditions with Kconfig dependency cones during `config-for` queries and verifies synthesized configurations through `make olddefconfig` and live compiler execution.

5.2 Construct Boundaries and Limitations

`kfold` models standard Kbuild semantics but faces natural boundaries when encountering unconstrained dynamic Make features:

- **Dynamic Code Evaluation (`eval`):** Makefiles that construct arbitrary new Make rules or variable assignments at runtime via `$(eval ...)` require full dynamic interpreter evaluation. In our Linux census, `eval` occurs rarely in core object selection.

- **Host-Dependent Shell Execution:** Kbuild Makefiles occasionally invoke host utilities via `$(shell ...)` to detect compiler capabilities (e.g., `as-instr` or `cc-option`). kfold evaluates shell calls concretely in the ambient host environment, which means analysis outcomes may reflect host toolchain capabilities.
- **Top-Level Build Orchestration:** Non-Linux firmware projects (such as U-Boot) occasionally specify directory selection in top-level packaging rules outside standard recursive child Makefile traversals. Extending entry-point specifications in project configuration files resolves these omissions; the entry-point ablation in RQ4 shows how much these settings matter.
- **Approximations in the Word Representation:** Guarded word sets do not preserve word order or duplicates, and concatenations whose alternatives exceed 64 combinations are collapsed (Table 2). Queries whose answer depends on order, such as `$(firstword ...)` over a conditional list, may therefore be approximate.
- **Architecture Coverage:** Our Linux analysis and all four Linux builds target x86 (i386 and x86-64). Other architectures need their own entry settings and have not been validated.

5.3 Practical Implications for Systems Development

Integrating kfold into continuous-integration pipelines offers substantial practical benefits for operating system development:

- **Preventing Testing Illusions in CI:** By checking whether modified files are active under test configurations, CI runners can detect when a pull request touches code that will not be compiled by existing test matrices.
- **Automated Test Configuration Synthesis:** Instead of relying on hand-crafted or randomly sampled configurations, test runners can use `config-for` to propose configurations close to a base configuration that select the changed files, then check them with `make olddefconfig` and a build.
- **Auditing Subsystem Blind Spots:** Kernel maintainers can use kfold to audit which objects none of a chosen set of configurations compiles, identifying code that those builds never test.

6 Related Work

Analyzing configurable software systems requires reasoning about variability across build systems, source-level conditional compilation, and configuration models [3, 6, 34].

6.1 Static Analysis of Kbuild and Build Variability

Kmax [13] is the pioneering static analysis framework for Kbuild Makefiles, demonstrating that whole-repository variability extraction is tractable without executing concrete builds. The Kmax ecosystem has established a powerful foundation for build-system verification, enabling configuration localization [15] and, together with the Kconfig analysis KCLAUSE, the unmet-dependency bug finder KISMET [29]. Static analysis of complex system build infrastructures remains a critically important yet historically under-explored area in software engineering, where decision representations play a central role. While Kmax explores Binary Decision Diagrams (BDDs) over complete string definitions, kfold investigates a complementary trade-off based on state folding and SMT solving.

In Kmax, variable values are represented as sets of alternative string definitions tagged with BDD presence conditions. kfold differs by decomposing target variables into independent whitespace-separated guarded words ($\sigma[n] = \{w \mapsto \phi_w\}$) whose conditions are Z3 formulas. Like Kmax's representation, and unlike path forking, this executes k independent conditional appends in $O(k)$ rather than $O(2^k)$ states, and it folds branch states at join points using fast-path identity merges

(§4.3). *kfold* also natively resolves composite members, models recursive Kbuild build semantics (need-builtin routes and container suppression), and closes over explicit rule prerequisites, directly powering interactive developer queries (why and config-for).

Other static build tools focus on different aspects of Makefile execution. Nguyen and Nguyen [28] symbolically execute Makefiles via path forking, which suffers exponential state explosion. Earlier approaches extracted Kbuild presence conditions with fuzzy parsing of Makefiles (KBuildMiner [4]) or by probing the build system with systematically varied configurations (GOLEM [10]). General build frameworks such as SYMAKE [32] and MAKAO [2] extract dependency graphs and analyze build structure, but do not model configuration-dependent object selection.

6.2 Variability Representations and State Merging

Variability-aware analyses process all variants of a configurable system without preprocessing individual variants [3, 22, 34]. In source code, TypeChef [18] and SuperC [14] parse C files containing conditional compilation directives (`#ifdef`) into variability-aware abstract syntax trees [17, 21]. Theoretical representations such as the Choice Calculus [12] formalize algebraic variation points. These source-level analyses complement *kfold*: while variability-aware parsers reason about preprocessor variability within translation units, they rely on build analysis to identify which translation units enter the compilation.

Symbolic execution engines traditionally explore execution paths by forking at branches [7, 8, 16, 19, 27, 31]. State merging mitigates path explosion by joining diverging paths at merge points into a unified symbolic state [20, 35]. *kfold* applies state merging to build scripts: it folds branch outcomes back into a single environment, tracking word-level guarded values and offloading propositional satisfiability and simplification to Z3 [9].

6.3 Configuration Modeling, Dead Code, and Testing

Kconfig specifies configuration spaces, dependencies, and default values across Linux, BusyBox, Barebox, U-Boot, and coreboot. Prior research studies Kconfig as a variability modeling language and extracts propositional constraints from Kconfig and Kbuild for feature modeling and consistency checking [5, 6, 10, 11, 29], while empirical studies analyze configuration co-evolution and reviewer complexity [24, 30]. Tartler et al. [33] developed Undertaker to detect dead code by checking Kconfig constraints against source-level `#ifdef` blocks. Other empirical studies analyze configuration defect root causes and mined constraints [1, 25, 26]. Like Kmax and GOLEM, *kfold* supplies the build-system link between Kconfig constraints and source-level translation units, here with object-level build semantics and developer queries on top. Furthermore, configuration sampling frameworks [23] can use *kfold*'s build conditions to synthesize focused test configurations for touched code.

7 Conclusion

Analyzing configuration-dependent build systems in large-scale system software like the Linux kernel is a vital yet notoriously challenging problem in software engineering and programming languages. The combination of thousands of configuration symbols, dynamic string manipulation, recursive directory traversals, and intricate Makefile dependencies creates an astronomical state space that defeats naive path exploration. Despite the fundamental role that build systems play in compiling modern operating systems, remarkably few formal symbolic methods have tackled this domain. Pioneering frameworks like Kmax demonstrated that static analysis of Kbuild Makefiles is possible through Binary Decision Diagrams, and this work introduces *kfold* to explore a complementary state-folding symbolic execution paradigm with SMT solving.

By representing target variables as independent guarded words and folding branch environments at conditional join points, *kfold* extracts reusable build conditions for the Linux x86 tree's 2,022

Makefile instances in 34 seconds. These build conditions turn silent compilation omissions into actionable developer insights, providing root-cause explanations through why and automated configuration synthesis for modified code through config-for. On four archived Linux x86 builds, kfold's predicted object sets equal the physical inventories; on four other configurable projects its precision ranges from 80.9% to 100%; and on generated Makefiles its cost stays flat where path forking explodes.

We hope that kfold and Kmax inspire renewed momentum and broader community participation in formal, symbolic reasoning for systems software build architectures. Future work includes extending whole-tree verification across heterogeneous hardware architectures, integrating automated configuration synthesis into live continuous-integration pipelines, and coupling build-condition extraction with whole-program symbolic verifiers. As configurable systems continue to expand in scale and complexity, a thriving ecosystem of build-aware analyses will be indispensable for advancing software testing, verification, and automated program repair across the software stack.

Abstract and title

Write these after the paper's emphasis and result scope are settled. The title above reflects both the condition analysis and the developer task, without making a universal exactness claim.

Project evidence used

- Core analysis: `src/ds.py`, `src/symexe.py`, `src/alg.py`, `src/objects.py`.
- Developer commands and tests: `src/cli/commands/`, `tests/test_cli_*.py`, `DEVTOOL_PLAN.md`.
- Physical, scaling, and baseline results (canonical run of 2026-09-25): `results/physical_validation.json`, `results/ablation.json`, `results/bench_scaling.json`, `results/kmax_physical_validation.json`. Superseded: `results/*_20260924.json`, `results/linux_gnu_make_validation.json`, `results/kfold_vs_kmax_comparison.json`; `evidence/claims.csv` still points to the superseded files and needs updating.
- Developer task records: `evidence/config_for_checkpoint_2b.json`, `evidence/blindspots_v6.6.json`, `evidence/lint_v6.6.json`.

References

- [1] Irit Abal, Claus Brabrand, and Andrzej Wasowski. 2014. 42 variability bugs in the alternative world: a study of Linux kernel flaws. In *Proceedings of the 2014 ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 379–390.
- [2] Bram Adams, Herman Tromp, Kris De Schutter, and Wolfgang De Meuter. 2007. MAKAO: An extensible framework for analyzing and re-engineering build systems. In *Proceedings of the 2007 IEEE International Conference on Software Maintenance (ICSM)*. 517–518.
- [3] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer Science & Business Media.
- [4] Thorsten Berger, Steven She, Rafael Lotufo, Krzysztof Czarnecki, and Andrzej Wasowski. 2010. Feature-to-Code Mapping in Two Large Product Lines. In *Software Product Lines: Going Beyond (SPLC 2010) (LNCS, Vol. 6287)*.
- [5] Thorsten Berger, Steven She, Rafael Lotufo, Andrzej Wasowski, and Krzysztof Czarnecki. 2010. Variability modeling in the real: a perspective from the operating systems domain. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 73–82. doi:10.1145/1858996.1859010
- [6] Thorsten Berger, Steven She, Rafael Lotufo, Andrzej Wasowski, and Krzysztof Czarnecki. 2013. A Study of Variability Models and Languages in the Systems Software Domain. *IEEE Transactions on Software Engineering* 39, 12 (2013), 1611–1640. doi:10.1109/TSE.2013.34
- [7] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224.
- [8] Lori A Clarke. 1976. A program testing system. *ACM '76: Proceedings of the 1976 Annual Conference (1976)*, 488–491.

- [9] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 337–340.
- [10] Christian Dietrich, Reinhard Tartler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. A Robust Approach for Variability Extraction from the Linux Build System. In *Proceedings of the 16th International Software Product Line Conference (SPLC)*.
- [11] Christian Dietrich, Reinhard Tartler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2018. Understanding Linux feature interactions through build and configuration. *ACM Transactions on Computer Systems (TOCS)* 35, 4 (2018), 1–28.
- [12] Martin Erwig and Eric Walkingshaw. 2011. The choice calculus: Language-independent variation programming. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 21, 1 (2011), 1–42.
- [13] Paul Gazzillo. 2017. Kmax: Finding All Configurations of Kbuild Makefiles Statically. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 279–290. doi:10.1145/3106237.3106283
- [14] Paul Gazzillo and Robert Grimm. 2012. SuperC: Parsing all of C by taming the preprocessor. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 339–350.
- [15] Paul Gazzillo, Ugur Koc, ThanhVu Nguyen, and Shiyi Wei. 2018. Localizing Configurations in Highly-Configurable Systems. In *Proceedings of the 22nd International Systems and Software Product Line Conference (SPLC), Challenge Track*. doi:10.1145/3233027.3236404
- [16] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation (PLDI)*. 213–223.
- [17] Christian Kästner, Paolo G. Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. Variability-Aware Parsing in the Presence of Lexical Macros and Conditional Compilation. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 805–824.
- [18] Andy Kenner, Christian Kästner, Steffen Haase, and Thomas Leich. 2010. TypeChef: Toward Type Checking #ifdef Variability in C. In *Proceedings of the 2nd International Workshop on Feature-Oriented Software Development (FOSD)*. 25–32. doi:10.1145/1868688.1868693
- [19] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [20] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient state merging in symbolic execution. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204.
- [21] Jörg Liebig, Sven Apel, Christian Lengauer, and Christian Kästner. 2011. Variability-aware static analysis at scale. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 105–115.
- [22] Jörg Liebig, Sven Apel, Christian Lengauer, Christian Kästner, and Michael Schulze. 2010. An analysis of the variability in forty preprocessor-based software product lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 105–114.
- [23] Flavio Medeiros, Marcio Ribeiro, Rohit Gheyi, Sven Apel, Christian Kästner, Bruno Ferreira, and Karine Santos. 2016. A comparison of 10 sampling algorithms for configurable systems. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 643–654.
- [24] Jean Melo, Claus Brabrand, and Andrzej Wasowski. 2016. How does variability affect code review? A study on the Linux kernel. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 17:1–17:26.
- [25] Sarah Nadi, Thorsten Berger, Christian Kästner, and Krzysztof Czarnecki. 2014. Mining configuration constraints: Static analyses and empirical evaluations. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 140–151.
- [26] Sarah Nadi, Thorsten Berger, Christian Kästner, and Krzysztof Czarnecki. 2014. Where do configuration defects come from? An empirical study of Linux Kernel configuration bugs. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 677–688.
- [27] ThanhVu Nguyen, Gregory M Kapfhammer, and Mary Lou Soffa. 2014. Using dynamic symbolic execution to improve regression test suites. In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. 1–10.
- [28] ThanhVu Nguyen and KimHao Nguyen. 2020. Using Symbolic Execution to Analyze Linux KBuild Makefiles. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 712–716. doi:10.1109/ICSME46990.2020.00078
- [29] Jeho Oh, Necip Fazıl Yıldırım, Julian Braha, and Paul Gazzillo. 2021. Finding Broken Linux Configuration Specifications by Statically Analyzing the Kconfig Language. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/3468264.3468578

- [30] Leonardo Passos, Ricardo Terra, Marco Tulio Valente, Krzysztof Czarnecki, and Andrzej Wasowski. 2013. Co-evolution of feature models and source code in Linux. In *Proceedings of the 17th International Software Product Line Conference (SPLC)*. 121–130.
- [31] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: a concolic unit testing engine for C. In *Proceedings of the 10th European Software Engineering Conference and 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*. 263–272.
- [32] Ahmed Tamrawi, Tung Thanh Nguyen, Jafar M Al-Kofahi, and Tien N Nguyen. 2012. SYMAKE: A tool for analyzing and transforming Makefiles. In *Proceedings of the 2012 ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–4.
- [33] Reinhard Tartler, Daniel Lohmann, Julio Sincero, and Wolfgang Schröder-Preikschat. 2011. Feature Consistency in Compile-Time-Configurable System Software: Facing the Linux 10,000 Feature Problem. In *Proceedings of the Sixth European Conference on Computer Systems (EuroSys)*. 47–60.
- [34] Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. 2014. Potential and limitations of analyses of configurable systems. *ACM Computing Surveys (CSUR)* 47, 1 (2014), 1–36.
- [35] Emina Torlak and Rastislav Bodik. 2014. A lightweight symbolic execution expressive solver-aided host. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 530–541.